PRACTICAL - 13

Part A — Practice Programs (Hour 1)

PROBLEM1	1. Generic containers: a generic Box with get/set and a Pair; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.
CODE	

```
class Box<T> {
   private T value;

   public void set(T value) {
      this.value = value;
   }

   public T get() {
      return value;
   }
}

class Pair<A, B> {
   private A first;
   private B second;

   public Pair(A first, B second) {
      this.first = first;
      this.second = second;
   }

   public A getFirst() {
      return first;
   }

   public B getSecond() {
      return second;
   }
}

public class GenericDemo {

   // Generic method
   public static <T> T firstElement(T[] array) {
      return array[0];
   }

   public static void main(String[] args) {
```

```
        // Box with Integer
        Box<Integer> intBox = new Box<>();
        intBox.set(100);

        // Box with String
        Box<String> stringBox = new Box<>();
        stringBox.set("Hello");

        System.out.println("Integer Box: " + intBox.get());
        System.out.println("String Box: " + stringBox.get());

        // Pair with different types
        Pair<String, Integer> student =
              new Pair<>("Om", 21);

        System.out.println("Name: " + student.getFirst());
        System.out.println("Age: " + student.getSecond());

        // Generic method
        String[] names = {"Dharmik", "Rahul", "Amit"};
        Integer[] numbers = {10, 20, 30};

        System.out.println("First name: " + firstElement(names));
        System.out.println("First number: " + firstElement(numbers));
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13> javac .\GenericDemo.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13> java GenericDemo
Integer Box: 100
String Box: Hello
Name: Om
Age: 21
First name: Om
First number: 10
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13>
```

PROBLEM2

2. Bounded stats: a generic method double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

CODE

```
public class BoundedStats {

    public static <N extends Number> double average(N[] values) {

        double sum = 0;

        for (N value : values) {
            sum += value.doubleValue();
        }
```

```
        return sum / values.length;
    }

    public static void main(String[] args) {

        Integer[] integers = {10, 20, 30, 40};
        Double[] doubles = {10.5, 20.5, 30.5};
        Long[] longs = {100L, 200L, 300L};

        System.out.println("Integer average: " + average(integers));
        System.out.println("Double average: " + average(doubles));
        System.out.println("Long average: " + average(longs));
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13> javac BoundedStats.java

PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13> java BoundedStats
Integer average: 25.0
Double average: 20.5
Long average: 200.0
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13>
```

PROBLEM3

3. (For Advanced Learners – home) Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.

CODE

```
class MyStack<T> {

    private T[] stack;
    private int top;

    @SuppressWarnings("unchecked")
    public MyStack(int size) {
        stack = (T[]) new Object[size];
        top = -1;
    }

    public void push(T value) {

        if (top == stack.length - 1) {
            System.out.println("Stack is full!");
            return;
        }

        stack[++top] = value;
    }
```

```
    public T pop() {

        if (top == -1) {
            System.out.println("Stack is empty!");
            return null;
        }

        return stack[top--];
    }

    public T peek() {

        if (top == -1) {
            System.out.println("Stack is empty!");
            return null;
        }

        return stack[top];
    }

    public boolean isEmpty() {
        return top == -1;
    }
}

public class GenericStackDemo {

    public static void main(String[] args) {

        // Integer stack
        MyStack<Integer> intStack = new MyStack<>(5);

        intStack.push(10);
        intStack.push(20);
        intStack.push(30);

        System.out.println("Integer stack pop: "
                + intStack.pop());

        System.out.println("Integer stack peek: "
                + intStack.peek());


        // String stack
        MyStack<String> stringStack = new MyStack<>(5);
```

```
        stringStack.push("Java");
        stringStack.push("Python");
        stringStack.push("C++");

        System.out.println("String stack pop: "
              + stringStack.pop());

        System.out.println("String stack peek: "
              + stringStack.peek());


        // Empty case
        while (!stringStack.isEmpty()) {
           stringStack.pop();
        }

        System.out.println("Trying to pop from empty stack:");
        stringStack.pop();
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13> javac .\GenericStackDemo.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13> java GenericStackDemo
Integer stack pop: 30
Integer stack peek: 20
String stack pop: C++
String stack peek: Python
Trying to pop from empty stack:
Stack is empty!
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 13>
```